

assumption that such terms typically may not be of interest to users and hence will not be queried. Or we can choose to omit terms that are the most frequent in the collection. Such terms probably don't have any meaningful information. For instance, in many collections of English language text, 'the' is the most commonly occurring term, but it does not help much in differentiating between the different documents. In other words, given Zipf's Law, we can choose to omit certain distinct terms in the collection from being maintained in the dictionary in order to keep the dictionary sizes reasonably small. Such a reduction in the number of terms in the dictionary belongs to one form of compression known as *lossy* compression, the other being *lossless* compression. Other types of *lossy* compression relevant to information retrieval are stemming, lemmatization and stop word removal techniques, which we have discussed in our earlier columns.

Note that unlike other use-cases where compression is performed typically to reduce the space requirements (for instance, people typically compress large picture files so that they occupy less disk size), in case of IR systems, compression is performed on the inverted index so that we can maintain the index in the main memory (since this improves the response time of the IR system to queries). Compression can be applied to both the dictionary and the postings list in the inverted index data structure.

Let us first look at dictionary compression. What would be a simple data structure for representing the dictionary? Recall that a dictionary consists of the vocabulary or the distinct terms in the collection. We can sort the vocabulary lexicographically and maintain it in an array of fixed width. What are the issues associated with this representation? Terms in the vocabulary can have extremely varying lengths. Well, it is obvious that by choosing a fixed width for each term in the vocabulary list, we could potentially end up wasting a large number of unused bytes in each term representation in the array. Consider the shortest term as 'the' and the longest term as 'thunderbird'. Since we allocated a fixed array in which each array element was allocated a size equal to the longest term in the collection, both 'the' and 'thunderbird' are allocated 11 bytes each. However, the term 'the' needs only three bytes; so the remaining eight bytes are a waste.

In order to avoid this wastage, we need to move away from fixed size array representation for the dictionary. One possibility is to consider the entire dictionary as a string of words in the vocabulary, in which the words are sorted lexicographically in the representational string for that dictionary. If we represent the entire dictionary as a single string, we need to find a way of locating the individual terms. One way to do this is to maintain a pointer for each term into the dictionary string. A term entry can be represented as the term frequency, the pointer to its corresponding postings list and a pointer into the dictionary where the term is actually

present. The current term ends where the next term begins and is pointed to by a different pointer. What is the extra overhead we have in the current representation? We now maintain term pointers for each term. Pointers typically take four bytes each, so for each term in the dictionary, we waste four bytes. Can we do better?

One possibility is to reduce the number of pointers we maintain. For instance, we can choose to maintain pointers for a group of terms. Each pointer points to the first term in a term group of size k (where k is the number of terms in the block). Now we have eliminated $k-1$ pointers for each block since we maintain the pointer only for the first term in the block. But we need to maintain information within each block to be able to identify where each term begins in the block. We do this by maintaining the term size explicitly at the beginning of each term. Assuming that we can represent the term size in one byte, for a block of size k , we have an overhead of k bytes for representing the term length. For each term in the block except the first term, we save approximately three bytes per term. Note that this representation is useful in terms of reducing the space requirements. But it incurs additional computational overhead, since it needs to search the block linearly to locate a specific term in the block (except for the first term in the block). I leave it as a question to readers to come up with a more compact representation of the dictionary.

My 'must-read book' for this month

This month's book suggestion comes from one of our readers, Rajeev, and his recommendation is very appropriate to this column—an excellent article on information retrieval titled 'Information retrieval: A survey' by Ed Greengrass available at <http://www.csee.umbc.edu/csee/research/cadip/readings/IR.report.120600.book.pdf>. Thank you, Rajeev for sharing this link.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in this column. It will help many readers who want to improve their software skills.

If you have any favourite programming questions or software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at [http://www.linkedin.com/groups?home=HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182"&gid=2339182"](http://www.linkedin.com/groups?home=HYPERLINK%20http%2F%2Fwww.linkedin.com%2Fgroups%3Fhome%3D2339182%26HYPERLINK%20http%2F%2Fwww.linkedin.com%2Fgroups%3Fhome%3D2339182%26gid%3D2339182%26gid%3D2339182)